Software Requirements Specification

AI-Based Smart Load Balancer

Document Type:	Software Requirements Specification (SRS)
Standard:	IEEE Std 830-1998
Version:	1.0
Date:	May 2026

Revision History

Version	Date	Author(s)	Description
1.0	May 2026	Project Team	Initial release of SRS document

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for the AI-Based Smart Load Balancer system. The document is prepared in accordance with IEEE Std 830-1998 and is intended for use by the development team, course instructor, and any future maintainers of the system. It serves as the authoritative reference for design, implementation, and verification activities.

1.2 Scope

The AI-Based Smart Load Balancer is a simulation system designed to distribute incoming client requests across three backend servers using three distinct strategies: Round Robin, Least Connections, and an AIassisted routing algorithm based on a Decision Tree Classifier. The system generates synthetic traffic, executes all three strategies against the same request workload, records per-request response times and per-server utilisation, and produces comparative visualisations.

The system does not implement real network communication; all server behaviour is simulated in memory. The scope is limited to the simulation prototype and its supporting machine-learning pipeline. **1.3 Definitions, Acronyms, and Abbreviations**

Term / Acronym	Definition
SRS	Software Requirements Specification
AI	Artificial Intelligence
ML	Machine Learning
DTC	Decision Tree Classifier — the supervised learning model used for AI-based routing
Round Robin (RR)	Load balancing strategy that distributes requests cyclically across servers
Least Connections (LC)	Load balancing strategy that routes each request to the server with the fewest active connections
Request	A simulated client task characterised by an identifier, an arrival time, and a processing duration
Response Time	The simulated time (in seconds) taken by a server to process a single request

Utilisation	The total number of requests handled by a server over a simulation run
Load Penalty	An additive delay applied to a server response time proportional to its current active-connection count
MetricTracker	Component that records and aggregates response-time metrics for a simulation run
model.pkl	Serialised Decision Tree Classifier produced by the training pipeline

1.4 References

- [1] IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications, IEEE, 1998.
- [2] Scikit-learn: Machine Learning in Python — DecisionTreeClassifier documentation, https://scikit-learn.org.

- [3] Python Software Foundation, Python 3.13 Documentation, https://docs.python.org/3.
- [4] Hunter, J.D., Matplotlib: A 2D graphics environment, Computing in Science & Engineering, vol. 9, no. 3, pp. 90-95, 2007.
- [5] Project Proposal: AI-Based Smart Load Balancer, Software Engineering Course, 2025.

1.5 Overview

Section 2 of this document provides an overall description of the product, including its context, major functions, user characteristics, constraints, and assumptions. Section 3 specifies the detailed functional and non-functional requirements together with system models including use-case descriptions, data-flow descriptions, and module interfaces. Section 4 contains appendices with the module structure and file inventory.

2. Overall Description

2.1 Product Perspective

The AI-Based Smart Load Balancer is a self-contained simulation prototype developed as a course project in Software Engineering. It is not part of a larger product suite and does not interface with live network infrastructure. The system operates entirely in a single Python process on a local workstation.

The architecture follows a layered design:

Layer	Component(s)	Responsibility
Simulation Entry	main.py	Orchestrates the full simulation pipeline

Traffic Generation	traffic/generator.py	Produces a list of synthetic request objects
Load Balancing	algorithms/round_robin.py algorithms/least_connection.py ml_model/predictor.py	Selects a target server per request using RR, LC, or AI strategy
Server Simulation	servers/server.py	Simulates request processing and tracks per-server state
ML Pipeline	ml_model/train.py ml_model/predictor.py	Trains and applies the Decision Tree Classifier
Metrics	metrics/tracker.py	Aggregates response-time data per simulation run
Visualisation	visualization/charts.py	Generates bar and line charts from collected metrics

2.2 Product Functions

The system provides the following major functions:

- F1 — Synthetic traffic generation: creates N configurable request objects with random arrival times and durations.
- F2 — Round Robin load balancing: distributes requests sequentially and cyclically across servers.
- F3 — Least Connections load balancing: routes each request to the server currently carrying the lowest active load.

- F4 — AI-assisted load balancing: uses a trained Decision Tree Classifier to predict the optimal server index given current load vectors.
- F5 — ML model training: generates synthetic training data, fits a DecisionTreeClassifier, and serialises the model to disk.
- F6 — Server simulation: models request processing with a load-dependent response-time formula.
- F7 — Metrics collection: records per-request response times and computes aggregate statistics.
- F8 — Comparative visualisation: renders average-response-time bar charts, server-utilisation bar charts, and response-time-over-time line charts.

2.3 User Characteristics

User Class	Description	Technical Level
Developer / Student	Runs main.py to execute simulations, modifies algorithm parameters, and inspects visualisation outputs.	Intermediate Python
Course Instructor	Reviews source code, documentation, and output charts to evaluate algorithmic correctness and engineering process.	General technical
Researcher (future)	May extend the system with additional algorithms or real traffic feeds.	Advanced Python / ML

2.4 Constraints

- C1 — The system must be implemented entirely in Python 3 using only standard-library and explicitly listed third-party packages (scikit-learn, NumPy, Matplotlib).
- C2 — No live network sockets may be used; all server communication is simulated in-process.
- C3 — The ML model must be a Decision Tree Classifier; substitution with other model types is out of scope for v1.0.
- C4 — The simulation must support exactly three backend servers in this version.
- C5 — Visualisation output files must be saved to the visualization/ directory as PNG images.
- C6 — The trained model must be persisted as ml_model/model.pkl using Python's pickle module.
- C7 — The system must run on a standard workstation without GPU acceleration.

2.5 Assumptions and Dependencies

- A1 — Python 3.10 or later is installed on the execution environment.
- A2 — The packages scikit-learn, NumPy, and Matplotlib are installed and importable.
- A3 — The working directory when invoking main.py is the project root (SE Project/).
- A4 — Sufficient disk write permission exists to create ml_model/model.pkl and visualization/*.png.
- A5 — Random seed is not fixed; simulation results will vary between runs, which is expected behaviour.
- A6 — The training data distribution (loads 0-20, request rate 1-50) is representative of the simulation workload.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

The system provides a command-line interface only. The user invokes the simulation by executing main.py from a terminal. Progress messages are printed to standard output in the format:

```
Training ML model...          (printed if model.pkl does not exist)
Round Robin    | Avg Response Time: X.XXXXs | Total Requests: N
Least Connections | Avg Response Time: X.XXXXs | Total Requests: N
AI Balancer    | Avg Response Time: X.XXXXs | Total Requests: N
```

No graphical user interface or web interface is provided in version 1.0.

3.1.2 Hardware Interfaces

The system has no hardware interface requirements beyond a standard workstation capable of running a CPython interpreter. File system read/write access is required for model persistence and PNG output.

3.1.3 Software Interfaces

Library	Version	Purpose
scikit-learn	>= 1.0	DecisionTreeClassifier training and inference (ml_model/train.py, predictor.py)
NumPy	>= 1.20	Feature array construction for ML inference (ml_model/predictor.py, train.py)
Matplotlib	>= 3.4	Chart generation (visualization/charts.py)
pickle	stdlib	Model serialisation / deserialisation
random	stdlib	Stochastic simulation of arrival times and response times
os	stdlib	Directory creation and file existence checks

3.1.4 Communication Interfaces

The system has no network communication interfaces. All inter-module communication occurs through direct Python function calls and shared in-memory objects.

3.2 Functional Requirements

Functional requirements are grouped by module. Each requirement is assigned a unique identifier of the form FR-XX.

3.2.1 Traffic Generator (traffic/generator.py)

ID	Requirement	Priority

FR-01	The system shall generate a configurable number N of request objects. Each request shall contain: a unique integer identifier (id), a simulated arrival time computed as i * Uniform(0.01, 0.05) seconds, and a simulated processing duration drawn from Uniform(0.1, 0.5) seconds.	High
FR-02	The default value of N shall be 300 requests per simulation run.	High
FR-03	The generator shall return requests as a Python list in ascending order of request identifier.	Medium

3.2.2 Server Simulation (servers/server.py)

ID	Requirement	Priority
FR-04	Each Server object shall maintain three internal counters: current_load (active connections), total_requests (cumulative handled count), and total_rps_time (cumulative response time).	High
FR-05	When handle_request(request_id) is called, the server shall: increment current_load; compute response time as Uniform(0.1, 0.5) + current_load * 0.05; add the response time to total_rps_time; decrement current_load; return the computed response time.	High
FR-06	The method utilization() shall return total_requests.	High

ID	Requirement	Priority
FR-07	The method avg_response_time() shall return total_rps_time / total_requests, or 0 if no requests have been handled.	Medium
FR-08	The method reset() shall set all counters to zero, enabling server reuse across simulation runs.	High

3.2.3 Round Robin Algorithm (algorithms/round_robin.py)

ID	Requirement	Priority
FR-09	The RoundRobin class shall accept a list of Server objects at construction time.	High
FR-10	The method get_server() shall return the server at the current cyclic index and then advance the index modulo the number of servers.	High
FR-11	The algorithm shall distribute exactly ceil(N/3) or floor(N/3) requests to each server over a run of N requests, differing by at most one request across servers.	High

3.2.4 Least Connections Algorithm (algorithms/least_connection.py)

ID	Requirement	Priority
FR-12	The LeastConnections class shall accept a list of Server objects at construction time.	High
FR-13	The method get_server() shall return the Server object whose current_load attribute is minimal at the moment of invocation.	High
FR-14	In the event of a tie in current_load, any tied server may be selected; tie-breaking order is not specified.	Low

3.2.5 AI Load Balancer (ml_model/predictor.py)

ID	Requirement	Priority
FR-15	The AILoadBalancer class shall load the serialised model from ml_model/model.pkl at construction time using Python's pickle module.	High
FR-16	The method get_server() shall: collect the current_load of all three servers; compute request_rate as the sum of loads plus one; construct a NumPy array of shape (1, 4) containing [load1, load2, load3, request_rate]; call model.predict() with this array; return the Server object at the predicted index.	High
FR-17	If model.pkl does not exist when AILoadBalancer is instantiated, the system shall raise a FileNotFoundError. The main script shall train the model before instantiating AILoadBalancer.	High

3.2.6 ML Model Training (ml_model/train.py)

ID	Requirement	Priority
FR-18	The function generate_training_data(samples) shall produce samples synthetic training examples. Each example shall consist of three independently drawn integer loads in [0, 20] and an integer request rate in [1, 50]. The label shall be the index of the minimum load value.	High
FR-19	The default number of training samples shall be 1000.	Medium
FR-20	The function train_model() shall fit a DecisionTreeClassifier on the generated data and persist the trained model to ml_model/model.pkl, creating the directory if necessary.	High
FR-21	The main script shall invoke train_model() automatically before the simulation if ml_model/model.pkl does not exist.	High

3.2.7 Metrics Tracker (metrics/tracker.py)

ID	Requirement	Priority
FR-22	The MetricTracker class shall maintain an in-memory list of response times recorded via the method record(response_time).	High
FR-23	The method average_reponse_time() shall return the arithmetic mean of all recorded response times, or 0 if no times have been recorded.	High
FR-24	The method total_requests() shall return the count of recorded response times.	High

3.2.8 Visualisation (visualization/charts.py)

ID	Requirement	Priority
FR-25	The function plot_avg_response_time(results) shall produce a bar chart comparing the average response time of Round Robin, Least Connections, and AI Balancer, saving it as visualization/avg_response_time.png.	High
FR-26	The function plot_server_utilization(results) shall produce a grouped bar chart showing the request count per server for each algorithm, saving it as visualization/server_utilzation.png.	High
FR-27	The function plot_response_time_over_time(rr_times, lc_times, ai_times) shall produce a line chart showing per-request response time across the 300-request workload for all three algorithms, saving it as visualization/response_over_time.png.	High
FR-28	All chart files shall be saved in PNG format at a resolution suitable for academic reporting (minimum 100 DPI).	Medium

3.3 Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	The full simulation pipeline (traffic generation, three algorithm runs, metric collection, and chart generation) for N = 300 requests and 1000 training samples shall complete within 30 seconds on a modern workstation (>= 2 GHz dual-core, >= 4 GB RAM).
NFR-02	Reliability	The system shall produce deterministic results for a fixed random seed. When no seed is set, the system shall complete without runtime errors on every execution.

NFR-03	Maintainability	Each module shall have a single, well-defined responsibility. No module shall exceed 100 lines of non-blank, non-comment code.
NFR-04	Portability	The system shall execute without modification on Windows, macOS, and Linux operating systems, given a compatible Python 3 installation.
NFR-05	Scalability	The architecture shall permit additional load-balancing algorithms to be added without modifying existing algorithm modules (open/closed principle). A new algorithm requires only creating a new class with a get_server() method.
NFR-06	Usability	The simulation shall be executable with a single command: python main.py. No configuration files or command-line arguments shall be required for the default run.
NFR-07	Testability	Each module shall be independently importable so that unit tests can exercise individual components without invoking the full simulation pipeline.
NFR-08	Data Integrity	Metric data shall not be shared between algorithm runs. Each algorithm shall operate on a fresh set of Server objects and a dedicated MetricTracker instance.

3.4 System Models

3.4.1 Use Case Descriptions **UC-01: Run Full Simulation**

Use Case ID:	UC-01
Name:	Run Full Simulation
Actor:	Developer / Student
Description:	The user executes the simulation pipeline, causing all three load balancing strategies to be compared against a common workload and visualisation outputs to be generated.
Preconditions:	Python 3 and required libraries are installed. Working directory is the project root.
Main Flow:	1. User executes: python main.py 2. System checks for model.pkl; trains model if absent. 3. System generates 300 synthetic requests. 4. System runs Round Robin simulation; records metrics. 5. System runs Least Connections simulation; records metrics. 6. System runs AI Balancer simulation; records metrics. 7. System prints comparison summary to stdout. 8. System generates and saves three PNG charts.

Postconditions:	Three PNG files exist in visualization/. model.pkl exists in ml_model/. Summary metrics are printed to stdout.
Exceptions:	E1: model.pkl cannot be written (disk full / permission denied) — simulation aborts with OS error. E2: Required library not installed — Python import error terminates execution.

UC-02: Retrain ML Model

Use Case ID:	UC-02
Name:	Retrain ML Model
Actor:	Developer
Description:	The developer explicitly retrains the Decision Tree model by running the training script directly.
Preconditions:	scikit-learn and NumPy are installed.
Main Flow:	1. Developer executes: python ml_model/train.py 2. System generates 1000 synthetic training samples. 3. System fits a DecisionTreeClassifier. 4. System saves model to ml_model/model.pkl. 5. System prints: Model trained and saved.
Postconditions:	ml_model/model.pkl is overwritten with a newly trained model.
Exceptions:	E1: Write permission denied — OSError is raised.

UC-03: Inspect Visualisation Output

Use Case ID:	UC-03
Name:	Inspect Visualisation Output
Actor:	Developer / Instructor
Description:	After a simulation run, the user opens the generated PNG charts to evaluate algorithm performance.

Preconditions:	UC-01 has been completed successfully.
Main Flow:	1. User navigates to visualization/ directory. 2. User opens avg_response_time.png, server_utilzation.png, and response_over_time.png using any image viewer.
	3. User interprets comparative performance data.
Postconditions:	No system state is modified.
Exceptions:	E1: Chart files do not exist — UC-01 must be run first.

3.4.2 Data Flow Description

The following describes the flow of data through the simulation pipeline for a single algorithm run. The same flow is repeated for each of the three algorithms.

Step	Source	Data	Destination
1	generator.py	List of 300 request dicts {id, arrival_time, duration}	main.py (requests list)
2	main.py	Request object (one per iteration)	Algorithm.get_server()
3	Algorithm	Selected Server object reference	main.py
4	main.py	request['id']	Server.handle_request()
5	Server	Computed response time (float, seconds)	main.py
6	main.py	Response time	MetricTracker.record()
7	main.py	All response times list	charts.py (plot functions)
8	main.py	results dict {avg_time, utilization[]}	charts.py (plot functions)
9	charts.py	PNG image data	visualization/*.png (disk)

3.4.3 Response Time Model

The simulated response time for each request processed by a server is computed as:

```
response_time = Uniform(0.1, 0.5)  +  current_load x 0.05
```

where current_load is the number of active connections on the server at the moment the request begins processing. Since the simulation is single-threaded and requests are processed sequentially within each algorithm run, current_load is typically 0 or 1 per request, yielding a practical response time range of [0.10, 0.55] seconds.

3.4.4 Module Interface Summary

Module	Public Interface	Returns
traffic.generator	generate_requests(n: int) -> list	List of n request dicts
servers.server	Server(name) handle_request(id) -> float utilization() -> int avg_response_time() -> float reset() -> None	Server instance Response time Request count Avg response time —

algorithms.round_robin	RoundRobin(servers) get_server() -> Server	RR instance Next server (cyclic)
algorithms.least_connection	LeastConnections(servers) get_server() -> Server	LC instance Least-loaded server
ml_model.predictor	AILoadBalancer(servers) get_server() -> Server	AI instance DTC-predicted server

ml_model.train	train_model() -> DecisionTreeClassifier generate_training_data(n) -> (X, y)	Fitted model Feature/label arrays

Module	Public Interface	Returns
metrics.tracker	MetricTracker() record(rt: float) average_reponse_time() -> float total_requests() -> int	Tracker instance — Mean RT Count
visualization.charts	plot_avg_response_time(results) plot_server_utilization(results) plot_response_time_over_time(rr, lc, ai)	Saves PNG Saves PNG Saves PNG

4. Appendices

Appendix A — File and Directory Inventory

Path	Type	Description
main.py	Python	Simulation entry point; orchestrates all algorithm runs and chart generation
algorithms/round_robin.py	Python	RoundRobin class implementing cyclic server selection
algorithms/least_connection.py	Python	LeastConnections class implementing min-load server selection
ml_model/predictor.py	Python	AILoadBalancer class wrapping the trained Decision Tree model
ml_model/train.py	Python	Training pipeline: data generation, model fitting, and serialisation

ml_model/model.pkl	Binary	Serialised DecisionTreeClassifier (generated at runtime)
metrics/tracker.py	Python	MetricTracker class for recording and aggregating response times
servers/server.py	Python	Server class simulating request processing with loaddependent latency
traffic/generator.py	Python	generate_requests() function producing synthetic workload
visualization/charts.py	Python	Chart generation functions using Matplotlib
visualization/avg_response_time.png	PNG	Bar chart: average response time per algorithm (generated at runtime)
visualization/server_utilzation.png	PNG	Grouped bar chart: requests per server per algorithm (generated at runtime)
visualization/response_over_time.png	PNG	Line chart: per-request response time over simulation run (generated at runtime)

Appendix B — Requirements Traceability Matrix

Requirement ID	Description (brief)	Source Module(s)	Priority
FR-01 – FR-03	Traffic generation	traffic/generator.py	High
FR-04 – FR-08	Server simulation	servers/server.py	High
FR-09 – FR-11	Round Robin algorithm	algorithms/round_robin.py	High

1.0

Requirement ID	Description (brief)	Source Module(s)	Priority
FR-12 – FR-14	Least Connections algorithm	algorithms/least_connection.py	High
FR-15 – FR-17	AI-based routing	ml_model/predictor.py	High
FR-18 – FR-21	ML training pipeline	ml_model/train.py	High
FR-22 – FR-24	Metrics collection	metrics/tracker.py	High
FR-25 – FR-28	Visualisation	visualization/charts.py	High
NFR-01	Execution performance	main.py (full pipeline)	Medium
NFR-02	Reliability / no runtime errors	All modules	High

NFR-03	Maintainability / SRP	All modules	Medium
NFR-04	Cross-platform portability	All modules	Medium
NFR-05	Extensibility (open/closed)	algorithms/	Medium
NFR-06	Single-command usability	main.py	High
NFR-07	Testability (importable modules)	All modules	Medium
NFR-08	Data isolation between runs	main.py	High